Summarize

Chapter 4. Architectural Decomposition

Monday, October 4, 10:04



Now that Addison and Austen had the go-ahead to move to a distributed architecture and break apart the monolithic Sysops Squad application, they needed to determine the best approach for how to get started.

"The application is so big I don't even know where to start. It's as big as an elephant!" exclaimed Addison.

"Well," said Austen. "How do you eat an elephant?"

"Ha, I've heard that joke before, Austen. One bite at a time, of course!" laughed Addison.

"Exactly. So let's use the same principle with the Sysops Squad application," said Austen. "Why don't we just start breaking it apart, one bite at a time? Remember how I said reporting was one of the things causing the application to freeze up? Maybe we should start there."

"That might be a good start," said Addison, "but what about the data? Just making reporting a separate service doesn't solve the problem. We'd need to break apart the data as well, or even create a separate reporting database with data pumps to feed it. I think that's too big of a bite to take starting out."

"You're right," said Austen. "Hey, what about the knowledge base functionality? That's fairly standalone and might be easier to extract."

"That's true. And what about the survey functionality? That should be easy to separate out as well," said Addison. "The problem is, I can't help feeling like we should be tackling this with more of a methodical approach rather than just eating the elephant bite by bite."

"Maybe Logan can give us some advice," said Austen.

Addison and Austen met with Logan to discuss some of the approaches they were considering for how to break apart the application. They explained to Logan that they wanted to start with the knowledge base and survey functionality but weren't sure what to do after that.

"The approach you're suggesting," said Logan, "is what is known as the Elephant Migration Anti-Pattern. Eating the elephant one bite at a time may seem like a good approach at the start, but in most cases it leads to an unstructured approach that results in a big ball of distributed mud, what some people also call a distributed monolith. I would not recommend that approach."

"So, what other approaches exist? Are there patterns we can use to break apart the application?" asked Addison.

"You need to take a holistic view of the application and apply either tactical forking or component-based decomposition," said Logan. "Those are the two most effective approaches I know of."

Addison and Austen looked at Logan. "But how do we know which one to use?"

Whereas architectural modularity describes the *why* for breaking apart a monolithic application, architectural decomposition describes the *how*. Breaking apart large, complex monolithic applications can be a complex and time-consuming undertaking, and it's important to know whether it is even feasible to begin such an effort and how to approach it.

Component-based decomposition and tactical forking are two common approaches for breaking apart monolithic applications. *Component-based decomposition* is an extraction approach that applies various refactoring patterns for refining and extracting components (the logical building blocks of an application) to form a distributed architecture in an incremental and controlled fashion. The *tactical forking* approach involves making replicas of an application and chipping away at the unwanted parts to form services, similar to the way a sculptor creates a beautiful work of art from a block of granite or marble.

Which approach is most effective? The answer to this question is, of course, *it depends*. One of the main factors in selecting a decomposition approach is how well the existing monolithic application code is structured. Do clear components and component boundaries exist within the codebase, or is the codebase largely an unstructured big ball of mud?

As the flowchart in Figure 4-1 illustrates, the first step in an architecture decomposition effort is to first determine whether the codebase is even decomposable. We cover this topic in detail in the next section. If the codebase is decomposable, the next step is to determine

if the source code is largely an unstructured mess with no clearly definable components. If that's the case, then tactical forking (see "Tactical Forking") is probably the right approach. However, if the source code files are structured in a way that combines like functionality within well-defined (or even loosely defined) components, then a component-based decomposition approach (see "Component-Based Decomposition") is the way to go.

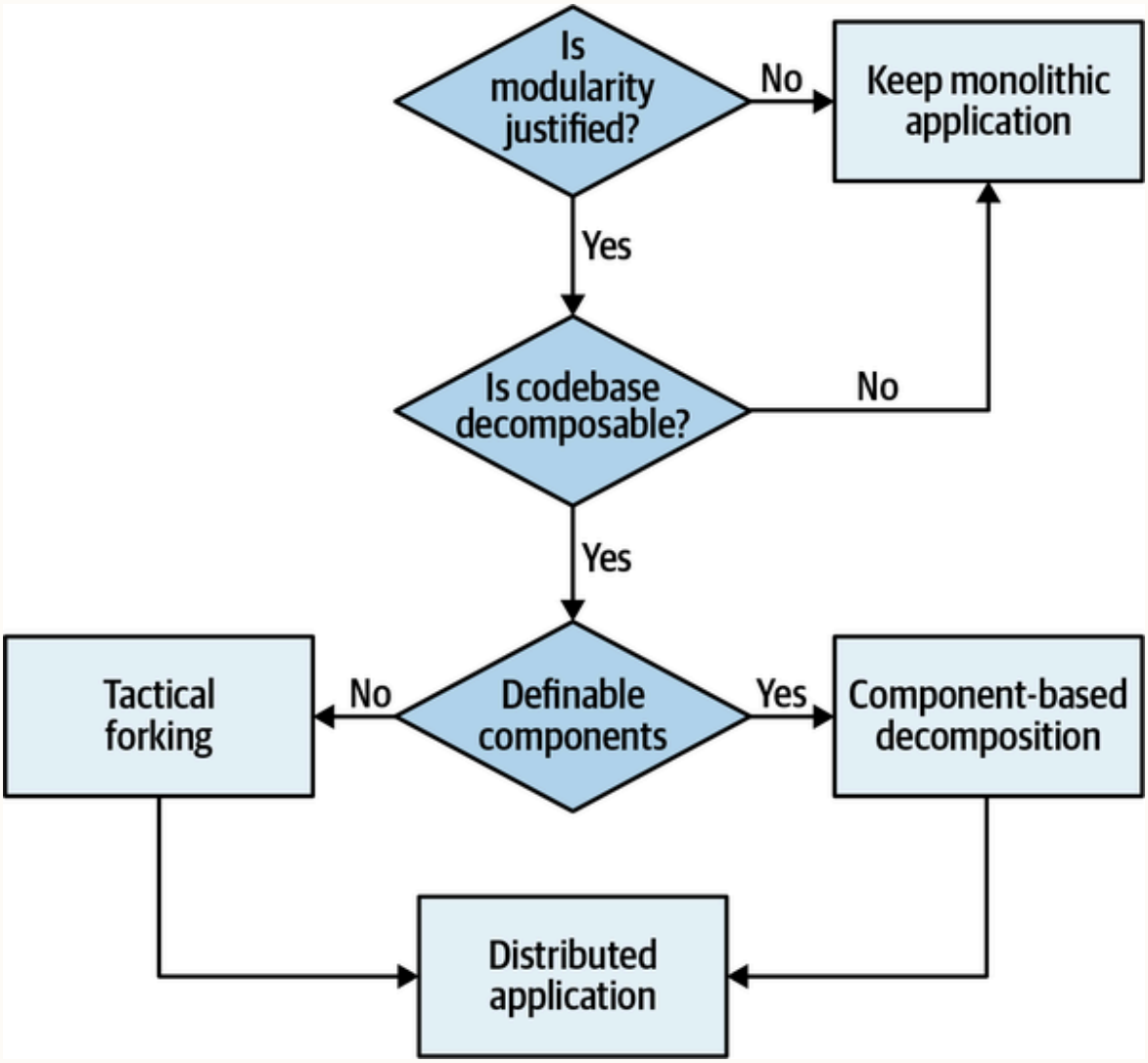


Figure 4-1. The decision tree for selecting a decomposition approach

We describe both of these approaches in this chapter, and then devote an entire chapter (Chapter 5) to describing each of the component-based decomposition patterns in detail.

Is the Codebase Decomposable?

What happens when a codebase lacks internal structure? Can it even be decomposed? Such software has a colloquial name—the Big Ball of Mud Anti-Pattern, coined by Brian Foote in a same-named essay in 1999. For example, a complex web application with event handlers wired directly to database calls and no modularity can be considered a Big Ball of Mud architecture. Generally, architects don't spend much time creating patterns for these kinds of systems; software architecture concerns internal structure, and these systems lack that defining feature.

Unfortunately, without careful governance, many software systems degrade into big balls of mud, leaving it to subsequent architects (or perhaps a despised former self) to repair. Step one in any architecture restructuring exercise requires an architect to determine a *plan* for the restructuring, which in turn requires the architect to understand the internal structure. The key question the architect must answer becomes is this codebase salvageable? In other words, is it a candidate for decomposition patterns, or is another approach more appropriate?

No single measure will determine whether a codebase has reasonable internal structure—that evaluation falls to one or more architects to determine. However, architects do have tools to help determine macro characteristics of a codebase, particularly coupling metrics, to help evaluate internal structure.

Afferent and Efferent Coupling

In 1979, Edward Yourdon and Larry Constantine published *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design* (Yourdon), defining many core concepts, including the metrics afferent and efferent coupling. *Afferent* coupling measures the number of *incoming* connections to a code artifact (component, class, function, and so on). *Efferent* coupling measures the *outgoing* connections to other code artifacts.

Note the value of just these two measures when changing the structure of a system. For example, when deconstructing a monolith into a distributed architecture, an architect will find shared classes such as `Address`. When building a monolith, it is common and encouraged for developers to reuse core concepts such as `Address`, but when pulling the monolith apart, now the architect must determine how many other parts of the system use this shared asset.

Virtually every platform has tools that allow architects to analyze the coupling characteristics of code in order to assist in restructuring, migrating, or understanding a codebase. Many tools exist for various platforms that provide a matrix view of class and/or component relationships, as illustrated in Figure 4-2.

In this example, the Eclipse plug-in provides a visualization of the output of JDepend, which includes coupling analysis per package, along with some aggregate metrics highlighted in the next section.

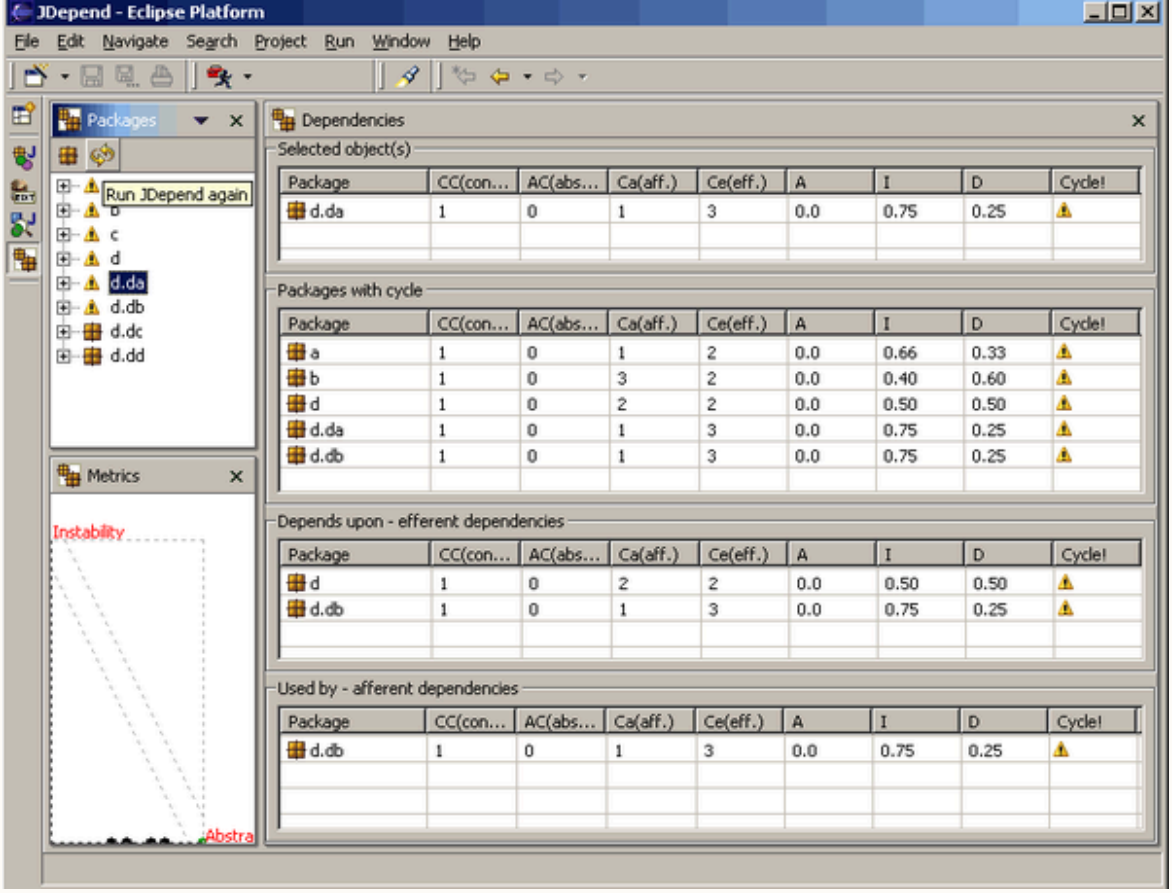


Figure 4-2. JDepend in Eclipse analysis view of coupling relationships

Abstractness and Instability

Robert Martin, a well-known figure in the software architecture world, created some derived metrics for a C++ book in the late 1990s that are applicable to any object-oriented language. These metrics—abstractness and instability—measure the balance of the internal characteristics of a codebase.

Abstractness is the ratio of abstract artifacts (abstract classes, interfaces, and so on) to concrete artifacts (implementation classes). It represents a measure of *abstract* versus *implementation*. Abstract elements are features of a codebase that allow developers to understand the overall function better. For example, a codebase consisting of a single `main()` method and 10,000 lines of code would score nearly zero on this metric and be quite hard to understand.

The formula for abstractness appears in Equation 4-1.

Equation 4-1. Abstractness

$$A = \frac{\sum m^a}{\sum m^c + \sum m^a}$$

In the equation, m^a represents *abstract* elements (interfaces or abstract classes) within the codebase, and m^c represents *concrete* elements. Architects calculate abstractness by calculating the ratio of the sum of abstract artifacts to the sum of the concrete ones.

Another derived metric, *instability*, is the ratio of efferent coupling to the sum of both efferent and afferent coupling, shown in Equation 4-2.

Equation 4-2. Instability

$$I = \frac{C^e}{C^e + C^a}$$

In the equation, C^e represents *efferent* (or outgoing) coupling, and C^a represents *afferent* (or incoming) coupling.

The instability metric determines the volatility of a codebase. A codebase that exhibits high degrees of instability breaks more easily when changed because of high coupling. Consider two scenarios, each with C^a of 2. For the first scenario, C^e = 0, yielding an instability score of zero. In the other scenario, C^e = 3, yielding an instability score of 3/5. Thus, the measure of instability for a component reflects how many potential changes might be forced by changes to related components. A component with an instability value near one is highly unstable, a value close to zero may be either stable or rigid: it is stable if the module or component contains mostly abstract elements, and rigid if it comprises mostly concrete elements. However, the trade-off for high stability is lack of reuse—if every component is self contained, duplication is likely.

A component with an *I* value close to 1, we can agree, is highly instable. However, a component with a value of I close to 0 may be either stable or rigid. However, if it contains mostly concrete elements, then it is rigid.

Thus, in general, it is important to look at the value of *I* and *A* together rather than in isolation. Hence the reason to consider the main sequence presented on the next page.

Distance from the Main Sequence

One of the few holistic metrics architects have for architectural structure is *distance from the main sequence*, a derived metric based on instability and abstractness, shown in Equation 4-3.

Equation 4-3. Distance from the main sequence

$$D = |A + I - 1|$$

In the equation, *A* = *abstractness* and *I* = *instability*.

The distance-from-the-main-sequence metric imagines an ideal relationship between abstractness and instability; components that fall near this idealized line exhibit a healthy

mixture of these two competing concerns. For example, graphing a particular component allows developers to calculate the distance-from-the-main-sequence metric, illustrated in Figure 4-3.

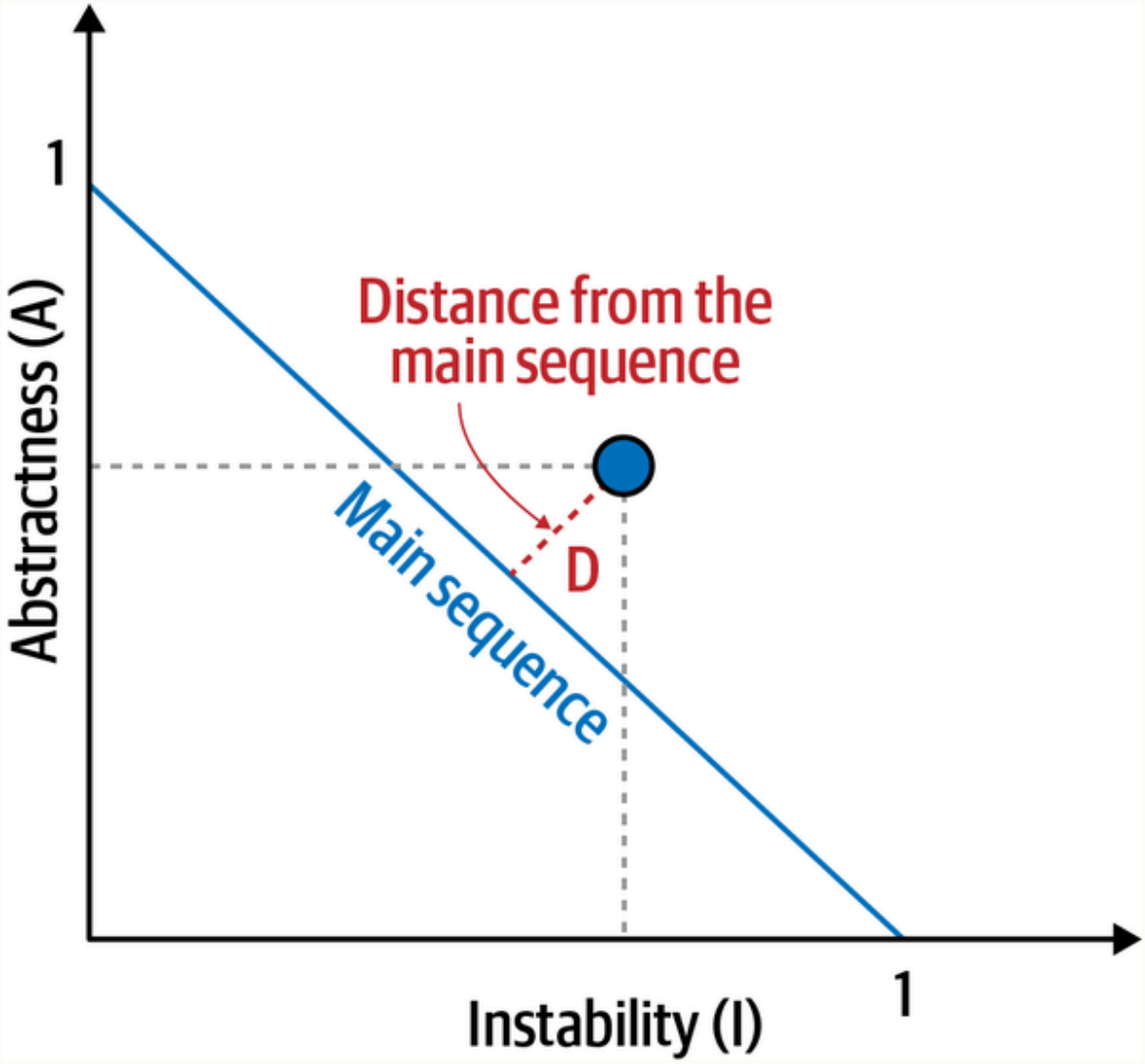


Figure 4-3. Normalized distance from the main sequence for a particular component

Developers graph the candidate component, then measure the distance from the idealized line. The closer to the line, the better balanced the component. Components that fall too far into the upper-right corner enter into what architects call the *zone of uselessness*: code that is too abstract becomes difficult to use. Conversely, code that falls into the lower-left corner enter the *zone of pain*: code with too much implementation and not enough abstraction becomes brittle and hard to maintain, illustrated in Figure 4-4.

Tools exist in many platforms to provide these measures, which assist architects when analyzing codebases because of unfamiliarity, migration, or technical debt assessment.

What does the distance-from-the-main-sequence metric tell architects looking to restructure applications? Just as in construction projects, moving a large structure that has a poor foundation presents risks. Similarly, if an architect aspires to restructure an application, improving the internal structure will make it easier to move the entity.

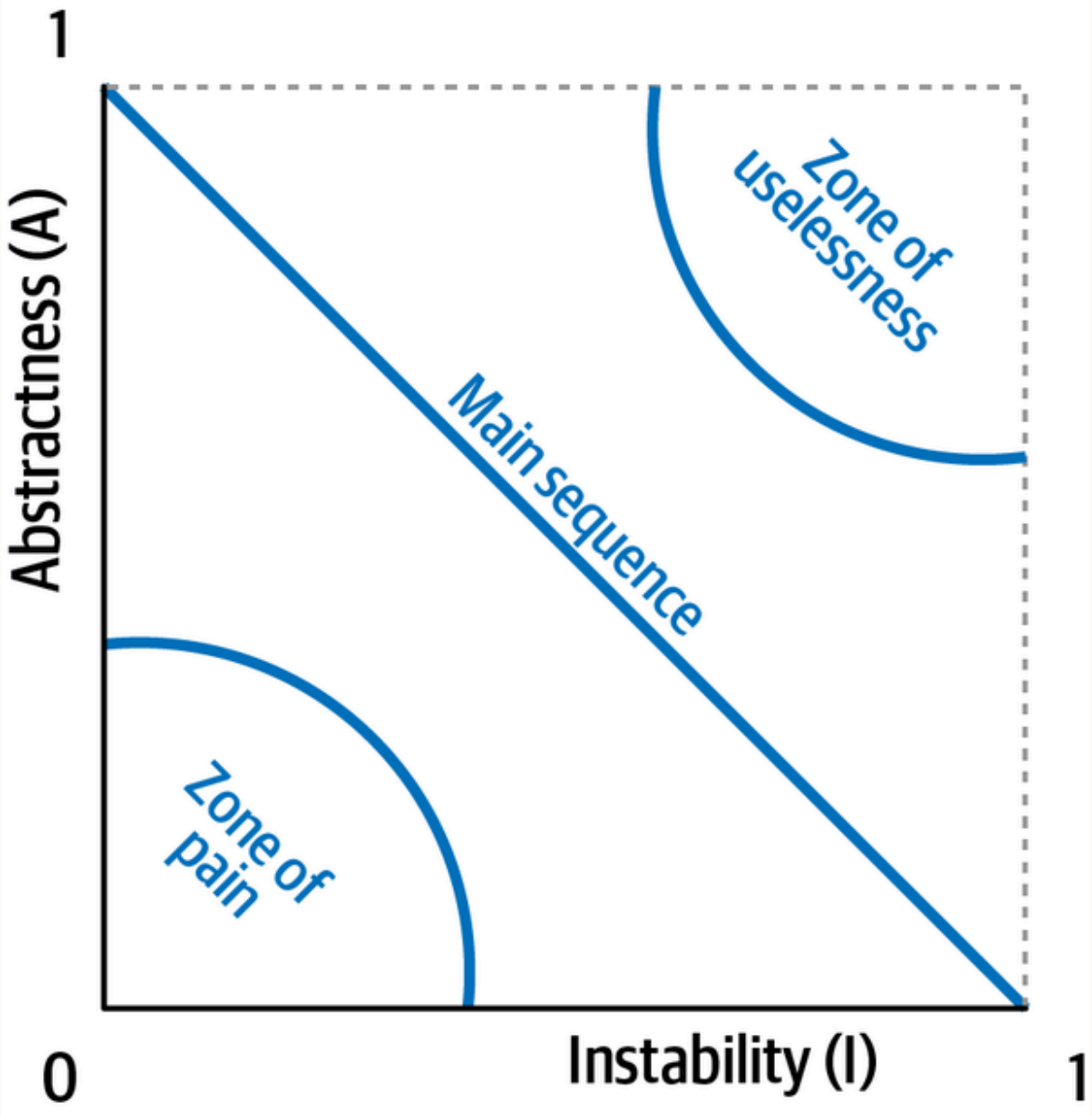


Figure 4-4. Zones of uselessness and pain

This metric also provides a good clue as to the balance of the internal structure. If an architect evaluates a codebase where many of the components fall into either the zones of uselessness or pain, perhaps it is not a good use of time to try to shore up the internal structure to the point where it can be repaired.

Following the flowchart in Figure 4-1, once an architect decides that the codebase is decomposable, the next step is to determine what approach to take to decompose the application. The following sections describe the two approaches for decomposing an application: *component-based decomposition* and *tactical forking*.

Component-Based Decomposition

It has been our experience that most of the difficulty and complexity involved with migrating monolithic applications to highly distributed architecture like microservices comes from poorly defined architectural components. Here we define a *component* as a building block of the application that has a well-defined role and responsibility in the system and a well-de-

fined set of operations. Components in most applications are manifested through namespaces or directory structures and are implemented through component files (or source files). For example, in Figure 4-5, the directory structure *penultimate/ss/ticket/assign* would represent a component called Ticket Assign with the namespace `penultimate.ss.ticket.assign` .

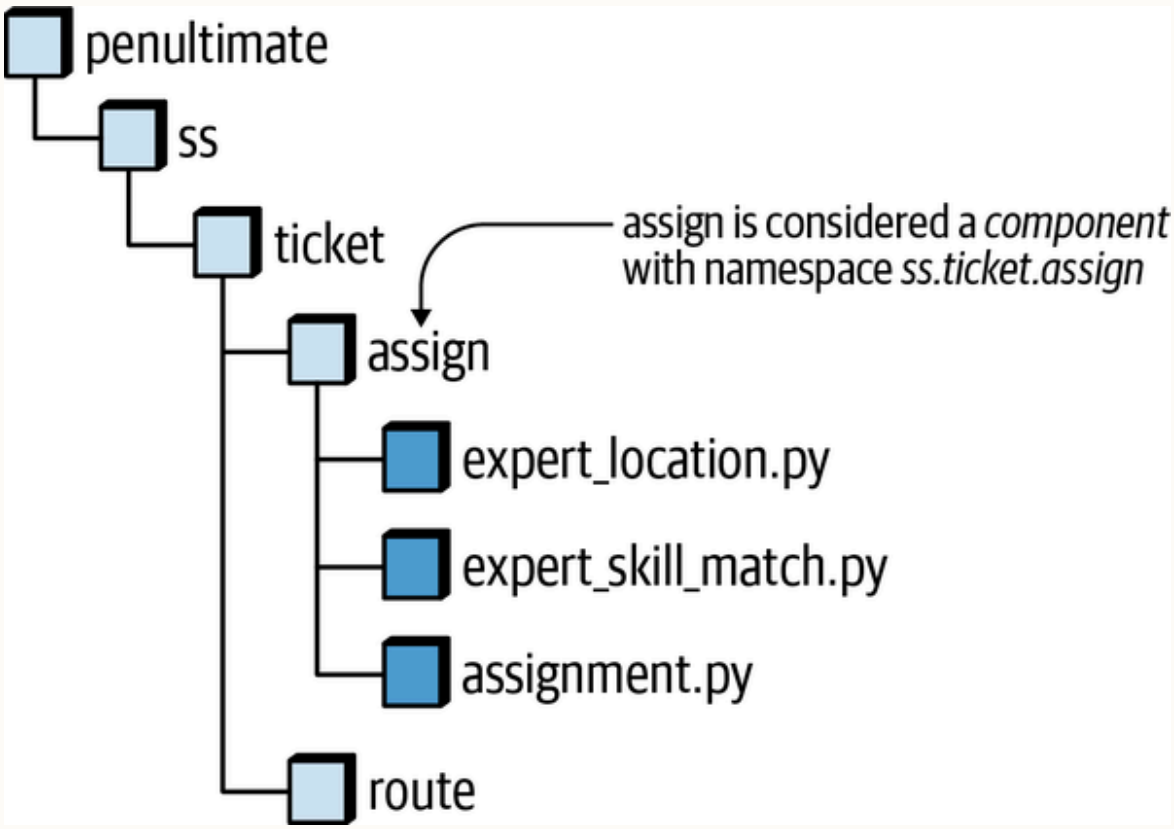


Figure 4-5. The directory structure of a codebase becomes the namespace of the component

---

TIP

When breaking monolithic applications into distributed architectures, build services from *components*, not individual classes.

Throughout many collective years of migrating monolithic applications to distributed architectures (such as microservices), we've developed a set of component-based decomposition patterns described in Chapter 5 that help prepare a monolithic application for migration. These patterns involve the refactoring of source code to arrive at a set of well-defined components that can eventually become services, easing the effort needed to migrate applications to distributed architectures.

These component-based decomposition patterns essentially enable the migration of a monolithic architecture to a service-based architecture, which is defined in Chapter 2 and described in more detail in *Fundamentals of Software Architecture*. Service-based architecture is a hybrid of the microservices architecture style where an application is broken into

domain services, which are coarse-grained, separately deployed services containing all of the business logic for a particular domain.

Moving to a service-based architecture is suitable as a final target or as a stepping-stone to microservices:

- As a stepping-stone, it allows an architect to determine which domains require further levels of granularity into microservices and which ones can remain as coarse-grained domain services (this decision is discussed in detail in Chapter 7).
- Service-based architecture does not require the database to be broken apart, therefore allowing architects to focus on the domain and functional partitioning prior to tackling database decomposition (discussed in detail in Chapter 6).
- Service-based architecture does not require any operational automation or containerization. Each domain service can be deployed using the same deployment artifact as the original application (such as an EAR file, WAR file, Assembly, and so on).
- The move to service-based architecture is a *technical one*, meaning it generally doesn't involve business stakeholders and doesn't require any change to the organization structure of the IT department nor the testing and deployment environments.

TIP

When migrating monolithic applications to microservices, consider moving to a service-based architecture first as a stepping-stone to microservices.

But what if the codebase is an unstructured big ball of mud and doesn't contain very many observable components? That's where tactical forking comes in.

Tactical Forking

The *tactical forking* pattern was named by Fausto De La Torre as a pragmatic approach to restructuring architectures that are basically big balls of mud.

Generally, when architects think about restructuring a codebase, they think of extracting pieces, as illustrated in Figure 4-6.

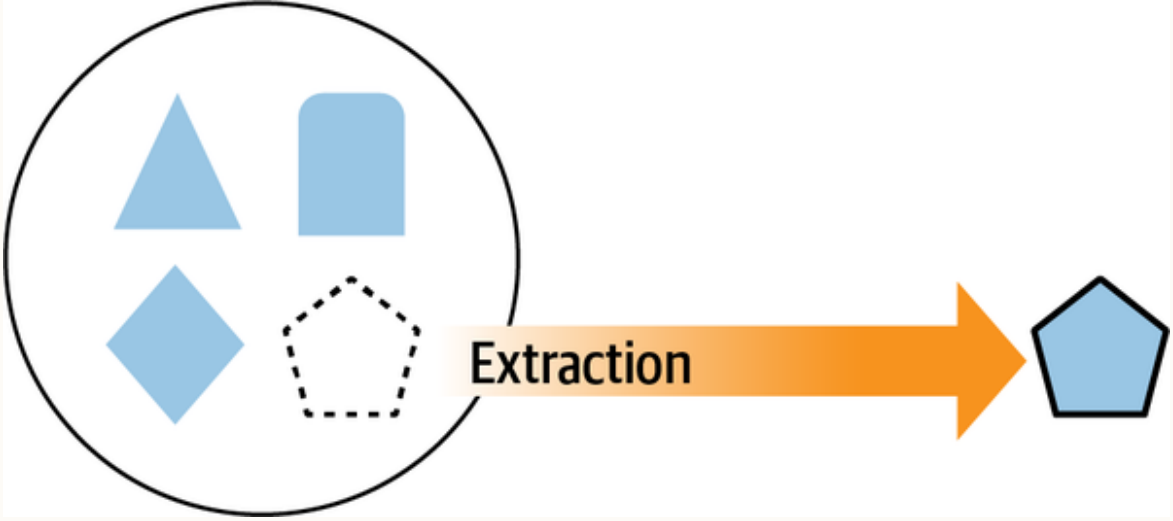


Figure 4-6. Extracting a part of a system

However, another way to think of isolating one part of a system involves *deleting* the parts no longer needed, as illustrated in Figure 4-7.

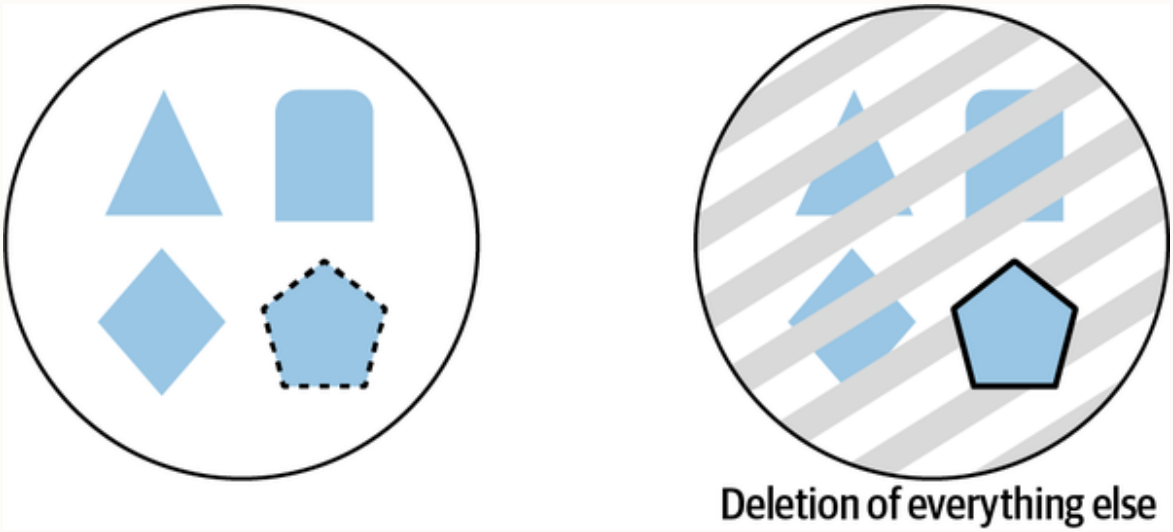


Figure 4-7. Deleting what's not wanted is another way to isolate parts of a system

In Figure 4-6, developers have to constantly deal with the exuberant strands of coupling that define this architecture; as they extract pieces, they discover that more and more of the monolith must come along because of dependencies. In Figure 4-7, developers delete what code isn't needed, but the dependencies remain, avoiding the constant unraveling effect of extraction.

The difference between *extraction* and *deletion* inspires the tactical forking pattern. For this decomposition approach, the system starts as a single monolithic application, as shown in Figure 4-8.

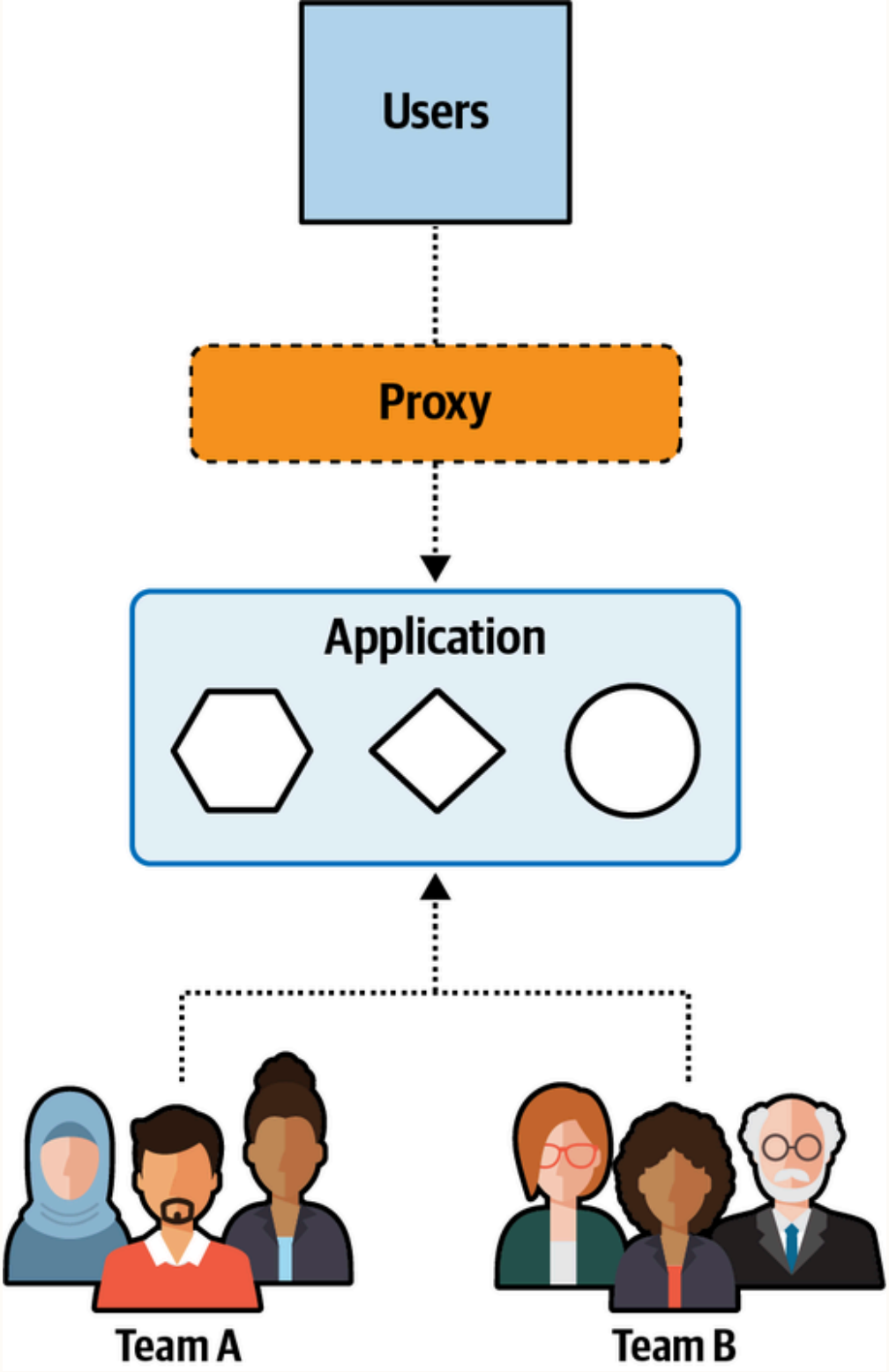


Figure 4-8. Before restructuring, a monolith includes several parts

This system consists of several domain behaviors (identified in the figure as simple geometric shapes) without much internal organization. In addition, in this scenario, the desired goal consists of two teams to create two services, one with the *hexagon* and *square* domain, and another with the *circle* domain, from the existing monolith.

The first step in tactical forking involves cloning the entire monolith, and giving each team a copy of the entire codebase, as illustrated in Figure 4-9.

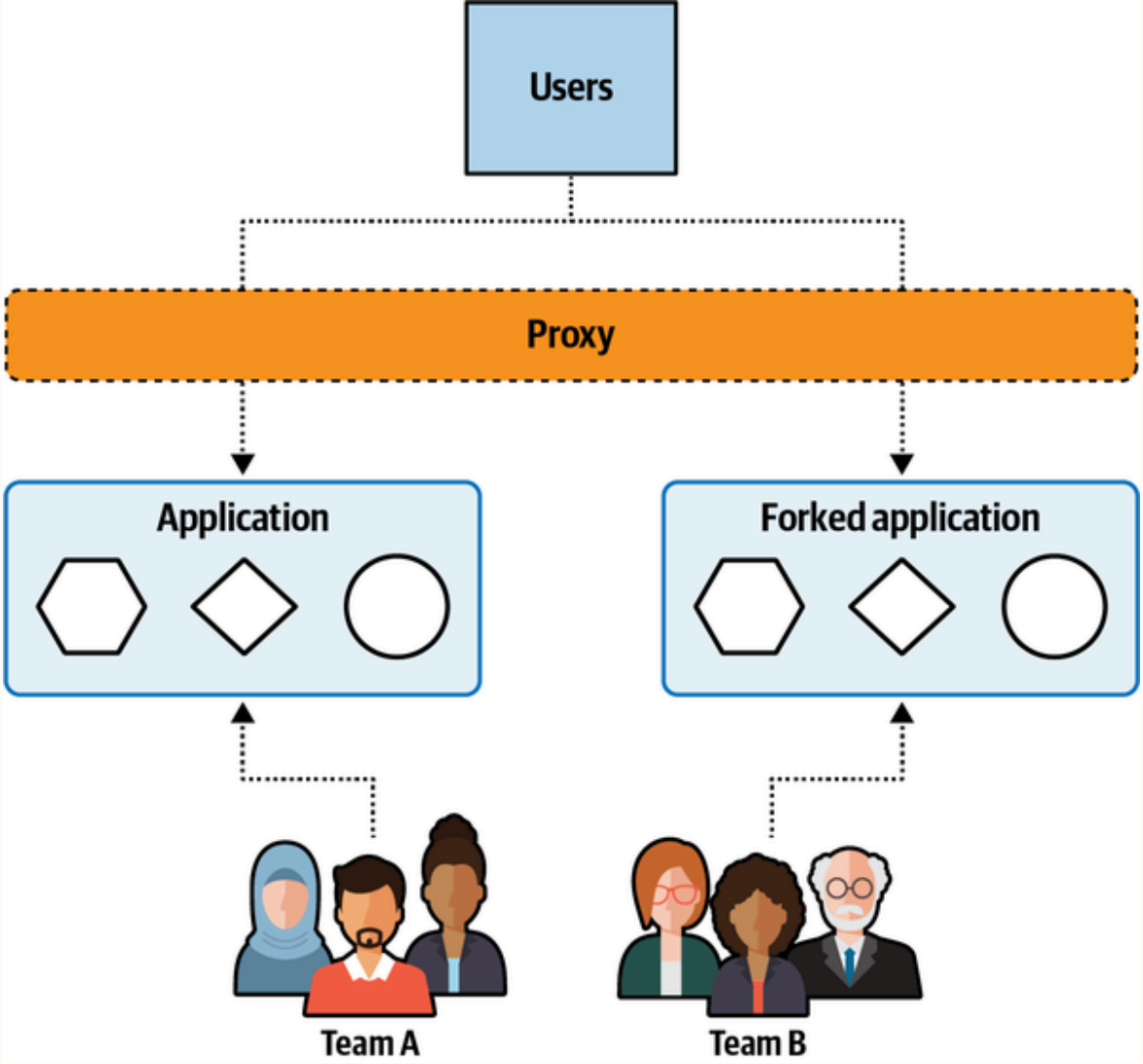


Figure 4-9. Step one clones the monolith

Each team receives a copy of the entire codebase, and they start *deleting* (as illustrated previously in Figure 4-7) the code they don't need rather than extract the desirable code. Developers often find this easier in a tightly coupled codebase because they don't have to worry about extracting the large number of dependencies that high coupling creates. Rather, in the *deletion* strategy, once functionality has been isolated, delete any code that doesn't break anything.

As the pattern continues to progress, teams begin to isolate the target portions, as shown in Figure 4-10. Then each team continues the gradual elimination of unwanted code.

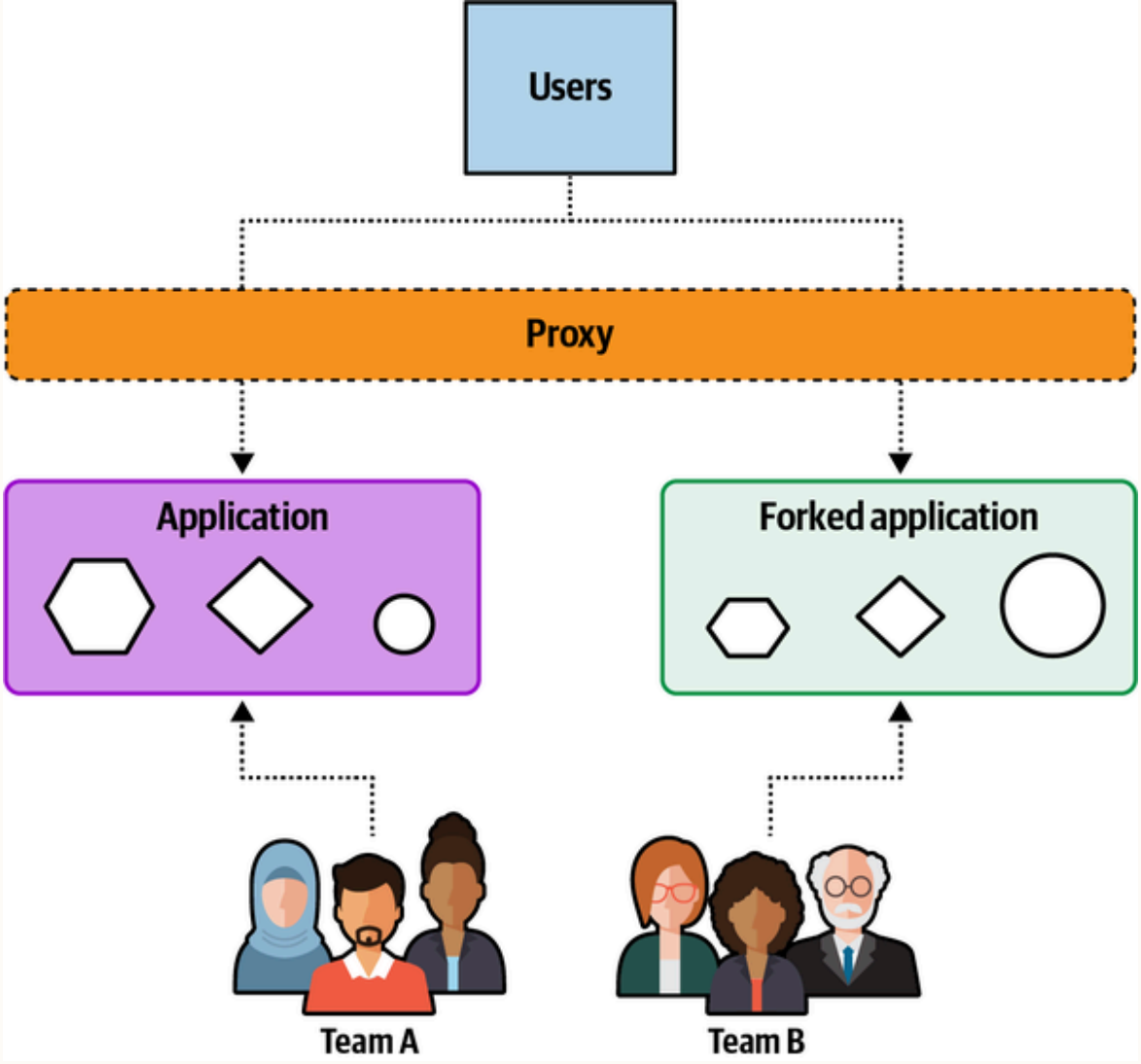


Figure 4-10. Teams constantly refactor to remove unwanted code

At the completion of the tactical forking pattern, teams have split the original monolithic application into two parts, preserving the coarse-grained structure of the behavior in each part, as illustrated in Figure 4-11.

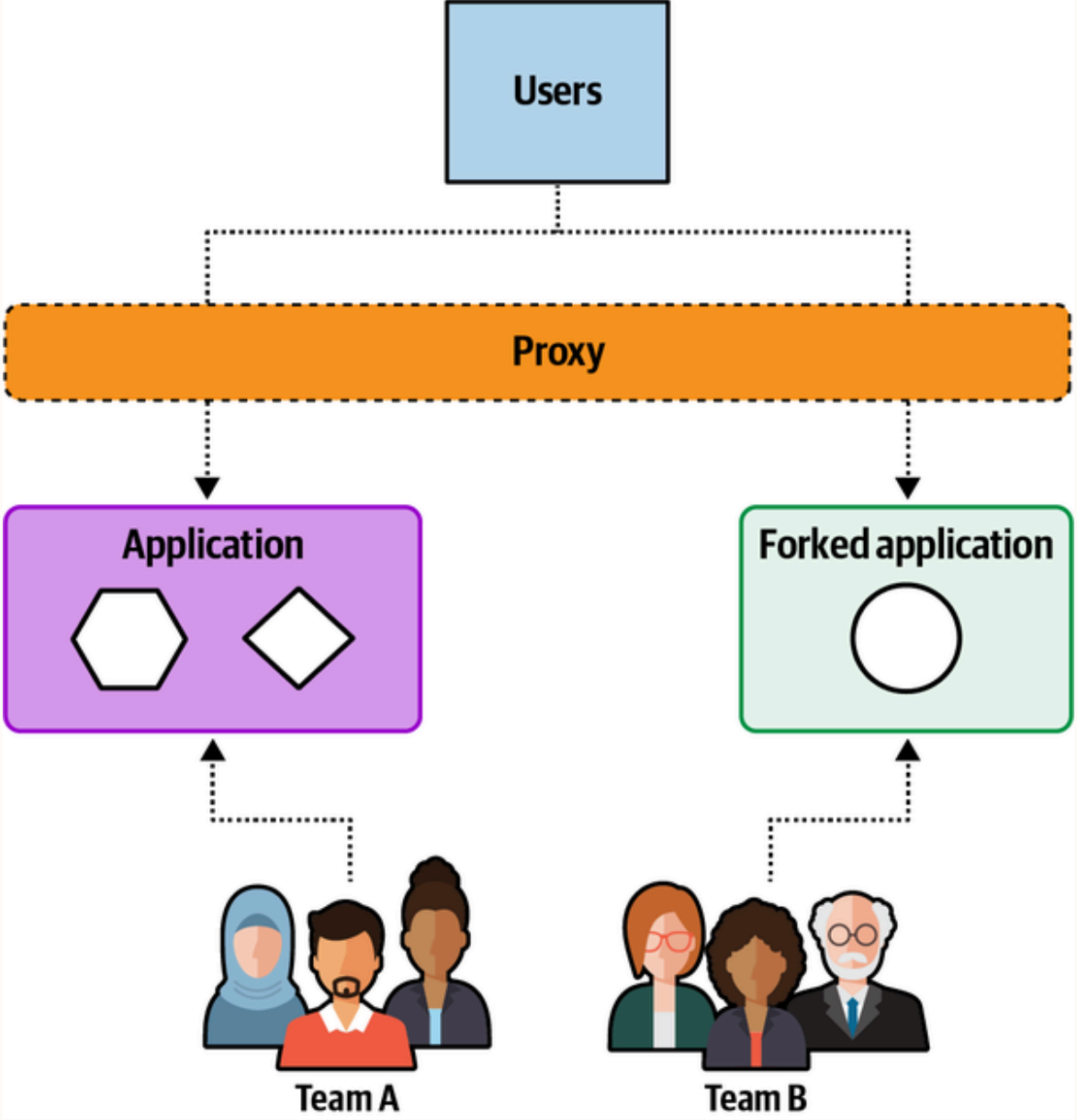


Figure 4-11. The end state of tactical forking features two services

Now the restructuring is complete, leaving two coarse-grained services as the result.

Trade-Offs

Tactical forking is a viable alternative to a more formal decomposition approach, most suited to codebases that have little or no internal structure. Like all practices in architecture, it has its share of trade-offs:

Benefits

- Teams can start working right away with virtually no up-front analysis.
- Developers find it easier to delete code rather than extract it. Extracting code from a chaotic codebase presents difficulties because of high coupling, whereas code not needed can be verified by compilation or simple testing.

Shortcomings

- The resulting services will likely still contain a large amount of mostly latent code left over from the monolith.
- Unless developers undertake additional efforts, the code inside the newly derived services won't be better than the chaotic code from the monolith—there's just less of it.
- Inconsistencies may occur between the naming of shared code and shared component files, resulting in difficultly identifying common code and keeping it consistent.

The name of this pattern is apt (as all good pattern names should be)—it provides a *tactical* rather than *strategic* approach for restructuring architectures, allowing teams to quickly migrate important or critical systems to the next generation (albeit in an unstructured way).

Sysops Squad Saga: Choosing a Decomposition Approach

Friday, October 29, 10:01



Now that Addison and Austen understood both approaches, they met in the main conference room to analyze the Sysops Squad application using the abstractness and instability metrics to determine which approach would be the most appropriate given their situation.

"Look at this," said Addison. "Most of the code lies along the main sequence. There are a few outliers of course, but I think we can conclude that it's feasible to break apart this application. So the next step is to determine which approach to use."

"I really like the tactical forking approach," said Austen. "It reminds me of famous sculptors, when asked how they were able to carve such beautiful works out of solid marble, who replied that they were merely removing the marble that wasn't supposed to be there. I feel like the Sysops Squad application could be my sculpture!"

"Hold on there, Michelangelo," said Addison. "First sports, and now sculpting? You need to make up your mind about what you like to spend your nonworking time on. The thing I don't like about the tactical forking approach is all the duplicate code and shared functionality within each service. Most of our problems have to do with maintainability, testability, and overall reliability. Can you imagine having to apply the same change to several different services at the same time? That would be a nightmare!"

"But how much shared functionality is there, really?" asked Austen.

“I’m not sure,” said Addison, “but I do know there’s quite a bit of shared code for the infrastructure stuff like logging and security, and I know a lot of the database calls are shared from the persistence layer of the application.”

Austen paused and thought about Addison’s argument for a bit. “Maybe you’re right. Since we have good component boundaries already defined, I’m OK with doing the slower component-based decomposition approach and giving up my sculpting career. But I’m not giving up sports!”

Addison and Austen came to an agreement that the component decomposition approach would be the appropriate one for the Sysops Squad application. Addison wrote an ADR for this decision, outlining the trade-offs and justification for the component-based decomposition approach.

ADR: Migration Using the Component-Based Decomposition Approach

Context
We will be breaking apart the monolithic Sysops Squad application into separately deployed services. The two approaches we considered for the migration to a distributed architecture were tactical forking and component-based decomposition.

Decision
We will use the component-based decomposition approach to migrate the existing monolithic Sysops Squad application to a distributed architecture.

The application has well-defined component boundaries, lending itself to the component-based decomposition approach.

This approach reduces the chance of having to maintain duplicate code within each service.

With the tactical forking approach, we would have to define the service boundaries up front to know how many forked applications to create. With the component-based decomposition approach, the service definitions will naturally emerge through component grouping.

Given the nature of the problems we are facing with the current application with regard to reliability, availability, scalability, and workflow, using the component-based decomposition approach provides a safer and more controlled incremental migration than the tactical forking approach does.

Consequences
The migration effort will likely take longer with the component-based decomposition approach than with tactical forking. However, we feel the justifications in the previous section outweigh this trade-off.

This approach allows the developers on the team to work collaboratively to identify shared functionality, component boundaries, and domain boundaries. Tactical forking would require us to break apart the team into smaller, separate teams for each forked application and increase the amount of coordination needed between the smaller teams.

‹ 3. Architectural Modularity

5. Component-Based Decomposition Patterns ›